Development of a Smart Autonomous Guided Vehicle Based Warehouse System with Integrated Robotic Arm for Material Picking and Transfer

A Project Report Submitted in Partial Fulfillment of the Requirements

for the Award of the Degree of

Bachelor of Technology

in

Mechanical Engineering / Robotics & Automation

Submitted by:

[Student Name]

(Faculty No.: XXXXXXXX)

Under the Guidance of:

[Supervisor Name]

Designation, Department of Mechanical Engineering

[University Name]

Department of Mechanical Engineering

[University Name]

Academic Year 2024–2025

Declaration

I hereby declare that this project report entitled *"Development of a Smart Autonomous Guided Vehicle Based Warehouse System with Integrated Robotic Arm for Material Picking and Transfer"* is the result of my own work and investigation, carried out under the supervision of [Supervisor Name], Department of Mechanical Engineering, [University Name]. This report has not been submitted elsewhere for the award of any other degree or diploma. All sources of information and literature used have been duly acknowledged.

Student Name: _____

Faculty No.: _____

Signature: _____

Date: _____

Certificate

This is to certify that the project report entitled *"Development of a Smart Autonomous Guided Vehicle Based Warehouse System with Integrated Robotic Arm for Material Picking and Transfer"* submitted by _____ (Faculty No. _____) is a bonafide record of work carried out under my direct supervision and guidance in partial fulfillment of the requirements for the degree of Bachelor of Technology in Mechanical Engineering during the academic year 2024–2025.

Project Guide:

_____

Designation: _____

Date: _____

Head of Department:

_____

Designation: _____

Date: _____

External Examiner: _____

Acknowledgement

The authors wish to express sincere gratitude to [Supervisor Name] for invaluable guidance, constant encouragement, constructive criticism, and unwavering support throughout the duration of this project. Their expertise in robotics and automation was instrumental in shaping this work.

Sincere thanks are extended to [Head of Department Name], Head of the Department of Mechanical Engineering, for providing the necessary infrastructure, laboratory facilities, and administrative support required to complete this project.

The authors are deeply grateful to all faculty members of the Department of Mechanical Engineering who contributed their knowledge and provided technical assistance whenever required during the course of this research.

The open-source communities behind ROS 2 (Open Robotics), Gazebo Simulator, MoveIt 2, Python, PyQt5, and Ubuntu Linux are gratefully acknowledged for providing the world-class tools that made this project possible. The Universal Robots and Robotiq communities are additionally acknowledged for their comprehensive hardware documentation and driver support.

Finally, profound gratitude is expressed to family and friends for their unconditional love, patience, and moral support throughout the academic journey.

Abstract

This project presents the design, implementation, and analysis of a smart Autonomous Guided Vehicle (AGV) based warehouse automation system integrated with a Universal Robots UR3 robotic arm for autonomous material picking and transfer. The system combines a differential-drive mobile platform (ATLAS AGV) with a 6-DOF collaborative manipulator to form a complete end-to-end warehouse automation solution capable of navigating to shelf locations, identifying targets via RFID, executing pick-and-place operations, and returning to a home docking station.

The ATLAS AGV employs an 8-channel infrared reflectance sensor array for line-following navigation at 0.4 m/s, achieving sub-5 mm tracking accuracy under a Proportional-Derivative (PD) control law. Navigation decision-making is governed by a 12-state Finite State Machine (FSM) with RFID-based shelf confirmation at 21 floor-embedded tag locations. The UR3 arm operates on a five-layer ROS 2 architecture spanning robot description, Gazebo Harmonic physics simulation, ros2_control real-time control, MoveIt 2 motion planning (OMPL and PILZ planners), and the MoveIt Task Constructor (MTC) compositional task framework with a Robotiq 2F-85 parallel gripper.

The integrated system is implemented on ROS 2 comprising 11 packages and a PyQt5-based industrial control centre. Key results demonstrate 100% mission completion rate, $\pm 3^{\circ}$ turn precision, sub-5 mm line tracking accuracy, and complete autonomous operation without human intervention. A simulation-to-physical conversion pathway is documented, confirming that 70% of the control codebase transfers directly to physical hardware. The combined minimum viable build is estimated at approximately $800 with a production-quality build at approximately $1,250.

Keywords: Autonomous Guided Vehicle, ROS 2, Differential Drive, Line Following, PID Control, RFID, Pick-and-Place, MoveIt 2, Warehouse Automation, Industry 4.0.

Contents

Notations, Symbols, and Abbreviations

Symbols

Symbol	Description	Unit
v	Linear velocity of robot centre	m/s
ω	Angular velocity of robot	rad/s
v_L	Left wheel linear velocity	m/s
v_R	Right wheel linear velocity	m/s
ω_L	Left wheel angular velocity	rad/s
ω_R	Right wheel angular velocity	rad/s
r	Wheel radius	m
L	Wheel track width (separation)	m
R	Instantaneous turning radius	m
θ	Robot heading angle	rad
K_p	Proportional controller gain	—
K_i	Integral controller gain	—
K_d	Derivative controller gain	—
$e(t)$	Control error signal	—
$u(t)$	Controller output	rad/s
τ	Motor torque	N·m
I_{xx}, I_{yy}, I_{zz}	Principal moments of inertia	kg·m^2
m	Robot mass	kg
g	Gravitational acceleration	m/s^2
μ	Coefficient of friction	—
F_N	Normal force	N
α	Angular acceleration	rad/s^2
ζ	Damping ratio	—
ω_n	Natural frequency	rad/s
σ	Standard deviation	varies

Symbol	Description	Unit
Δs	Incremental distance	m
$\Delta\theta$	Incremental angle	rad
w_i	Sensor position weight	—
s_i	Binary sensor output	{0,1}
$a, d, \alpha_{\mathrm{DH}}, \theta$	Denavit-Hartenberg parameters	m, m, rad, rad

Abbreviations

Acronym	Full Form
AGV	Automated Guided Vehicle
AMR	Autonomous Mobile Robot
CPR	Counts Per Revolution
DDS	Data Distribution Service
DH	Denavit-Hartenberg
DoF	Degrees of Freedom
EKF	Extended Kalman Filter
FSM	Finite State Machine
GPIO	General Purpose Input/Output
GUI	Graphical User Interface
HMI	Human-Machine Interface
I2C	Inter-Integrated Circuit
ICR	Instantaneous Centre of Rotation
IMU	Inertial Measurement Unit
IR	Infrared
LiDAR	Light Detection and Ranging
LLM	Large Language Model
MTC	MoveIt Task Constructor
OMPL	Open Motion Planning Library
PD	Proportional-Derivative
PID	Proportional-Integral-Derivative

Acronym	Full Form
PWM	Pulse Width Modulation
QoS	Quality of Service
RFID	Radio Frequency Identification
ROS2	Robot Operating System 2
RPM	Revolutions Per Minute
SAC	Soft Actor-Critic
SLAM	Simultaneous Localization and Mapping
SPI	Serial Peripheral Interface
TCP	Tool Centre Point
TF	Transform Frame
UART	Universal Asynchronous Receiver-Transmitter
URDF	Unified Robot Description Format
WMS	Warehouse Management System
YOLO	You Only Look Once

List of Tables

List of Figures

1. Introduction and Literature Review

1.1.1 Project Overview

Modern warehouses demand high-throughput, error-free material handling with minimal human intervention. Traditional warehouse operations, where workers spend 60–70% of their time on item retrieval walks, create significant operational inefficiencies [1]. Autonomous Guided Vehicles (AGVs) address these inefficiencies through continuous, precise material transport, while integrated robotic arms extend AGV capabilities to the physical manipulation of inventory items. The combination of guided mobile navigation with dexterous arm manipulation represents a complete material-handling automation solution aligned with the goals of Industry 4.0.

This project presents the development of the *ATLAS Smart Warehouse AGV*, a ROS 2-based autonomous mobile robot integrated with a Universal Robots UR3 6-DOF collaborative arm and a Robotiq 2F-85 parallel gripper. The system is capable of: receiving pick missions via a control GUI; navigating autonomously to target shelf locations using line-following and RFID identification; executing structured pick-and-place operations using motion-planned arm trajectories; and returning to a home docking station. The entire system operates within a simulated Gazebo warehouse environment with a clearly documented pathway for physical hardware deployment.

1.1.2 Problem Statement

Contemporary warehouses face critical operational challenges: human labour accounts for a disproportionate share of material retrieval time; manual picking is error-prone; round-the-clock operation is constrained by worker availability; and human-forklift interaction zones pose safety hazards. Existing commercial AGV solutions, such as the Amazon Robotics Kiva and MiR 250 systems, carry unit costs of $25,000 to $30,000 and above, and are closed systems that do not permit research-level modification [2]. An open-source platform combining mobile AGV navigation with a collaborative robotic arm for integrated picking is therefore needed to bridge the gap between academic research and industrial deployment. This project addresses that need by combining a differential-drive AGV with a six-axis manipulator on the ROS 2 middleware

framework.

1.1.3 Objectives

The specific objectives of this project are as follows:

1. Design and implement a differential-drive AGV platform with accurate kinematic modelling for warehouse navigation.

2. Implement line-following navigation using an 8-channel infrared reflectance sensor array and a PD controller.

3. Implement RFID-based shelf identification for position confirmation at 21 warehouse locations.

4. Design and integrate a 12-state Finite State Machine for complete autonomous mission lifecycle management.

5. Integrate a UR3 robotic arm with a Robotiq 2F-85 gripper for autonomous pick-and-place operations.

6. Implement MoveIt 2-based motion planning using OMPL and PILZ planners within the MoveIt Task Constructor (MTC) framework.

7. Develop a PyQt5 industrial control centre GUI for mission dispatch, real-time telemetry, and emergency controls.

8. Demonstrate complete autonomous warehouse operation in a Gazebo simulation environment.

9. Document the simulation-to-physical deployment conversion pathway.

1.1.4 Scope

The project covers the complete robotics stack for a warehouse AGV with a robotic arm. The AGV subsystem scope includes mechanical design (URDF/Xacro model), sensor systems (line sensor, IMU, RFID), PD-based control, mission state machine, and Gazebo Classic 11 simulation. The arm subsystem scope includes UR3 URDF modelling, Gazebo Harmonic physics simulation, ros2_control integration, MoveIt 2 motion planning, MTC-based compositional task

planning, and Robotiq 2F-85 gripper integration. The integrated scope includes a unified ROS 2 communication architecture, a shared GUI, and a documented hardware deployment guide.

1.1.5 Methodology

This project follows a simulation-first development methodology structured in six phases: (1) Requirements analysis — defining warehouse layout, shelf positions, arm workspace, and mission types; (2) System architecture — designing the ROS 2 package structure, node communication graph, and hardware interface strategy; (3) Component development — implementing AGV navigation, arm control, and GUI nodes independently; (4) Integration testing — combining all nodes and verifying inter-system communication; (5) System validation — executing complete missions from pick dispatch to home return; and (6) Performance analysis — measuring timing, accuracy, reliability, and arm positioning metrics.

1.1.6 Literature Review

1.1.6.1.6.1 Automated Guided Vehicles in Industry

AGVs have evolved significantly since their introduction in the 1950s. As documented by De Ryck et al. [1], AGV technology can be categorised across five generations: first-generation buried wire induction systems (1950s, Barrett Electronics); second-generation magnetic tape guidance (1980s, Daifuku); third-generation laser triangulation SLAM (2000s, SICK NAV350); fourth-generation natural feature SLAM (2010s, MiR, OTTO); and fifth-generation AI-based fleet coordination systems (2020s, Amazon Robotics). The ATLAS project implements a second-to-third generation hybrid — line-following with RFID augmentation — which remains the dominant method in structured warehouse environments owing to its reliability, low cost, and deterministic behaviour.

Azadeh et al. [2] report that line and tape guidance systems account for approximately 60% of all installed AGV systems globally as of 2023, attributable to deterministic behaviour suitable for safety certification, zero mapping requirements for immediate deployment, no sensor degradation, low computational requirements, and predictable paths suited to fixed warehouse layouts.

1.1.6.1.6.2 Differential Drive Kinematics

The differential drive mechanism is the most common mobile robot drive system due to its zero-turning-radius capability, simple mechanical construction, and straightforward kinematic

model [3]. Siegwart et al. [3] derive the forward kinematics from individual wheel velocities to robot body velocity and the inverse kinematics from desired robot velocity to wheel commands, both of which are implemented in this project.

1.1.6.1.6.3 PID Control for Line Following

PID control is established as the industry standard for line-following AGVs owing to its mathematical simplicity, well-understood tuning procedures, and real-time capability [4]. The ATLAS system uses a PD controller ($K_I = 0$) as recommended for line-following applications to eliminate integral windup issues at junction transitions [4].

1.1.6.1.6.4 ROS 2 as Robot Middleware

ROS 2 was selected over ROS 1 based on the following advantages documented by Open Robotics [5]: real-time capability through DDS-based communication; decentralisation eliminating the rosmaster single point of failure; production-quality Quality-of-Service (QoS) policies; multi-platform support; and Long-Term Support through 2027 for the Humble release.

1.1.6.1.6.5 RFID in Warehouse Automation

RFID provides position confirmation in structured environments without line-of-sight requirements, with read-while-moving capability, unique identification per location, and passive tags requiring no power or maintenance [6]. Finkenzeller [6] documents the 125 kHz passive RFID systems used in warehouse applications, forming the basis for the RDM6300 reader hardware selected for physical deployment.

1.1.6.1.6.6 Robotic Arm Manipulation with ROS 2

The MoveIt 2 motion planning framework is the standard ROS 2 interface for robotic arm manipulation. Corke [7] provides the mathematical foundations for serial manipulator kinematics including the Denavit-Hartenberg (DH) parameter convention used for the UR3 arm in this project. The Open Motion Planning Library (OMPL) implements sampling-based planners including RRTConnect, which serves as the primary planner for free-space arm motions [8]. The PILZ industrial motion planner provides deterministic Cartesian interpolation (LIN motions) for approach and retreat phases of the pick-and-place cycle.

1.1.6.1.6.7 Industry 4.0 and Smart Warehousing

Wurman et al. [9] describe the architecture of the Amazon Kiva system as involving mobile robots, overhead pick stations, and centralised fleet management. The ATLAS project addresses

a comparable functional scope at significantly reduced cost using open-source middleware and commercial off-the-shelf hardware. The integration of a collaborative robotic arm additionally addresses the “last metre” problem in warehouse automation — the physical transfer of individual items from shelving — which Amazon Kiva and MiR systems do not address autonomously.

2. Problem Formulation

2.2.1 Core Problem Definition

The central engineering problem addressed by this project is the design of an integrated autonomous system capable of performing a complete warehouse pick-and-transfer cycle without human intervention. The system must: (i) navigate a known warehouse floor layout to a commanded shelf location using reliable, deterministic guidance; (ii) confirm its arrival at the correct shelf via a secondary identification mechanism; (iii) execute a structured pick operation at the shelf using a robotic arm with appropriate motion planning; (iv) return to a home docking station with the picked item; and (v) repeat the cycle in response to queued missions dispatched from an operator interface.

2.2.2 Technical Sub-Problems

Navigation problem: The AGV must follow a tape-based guide path with sub-5 mm lateral error at 0.4 m/s, detect junction points for directional decisions, and execute precise 90° and 180° turns.

Localisation problem: Without SLAM, the system must determine its shelf position using junction counting combined with RFID tag detection, with appropriate hysteresis to prevent false triggers.

Mission management problem: A state machine must manage the complete lifecycle including idle waiting, navigation, picking, returning, and docking, with emergency stop and reset capabilities at all times.

Manipulation problem: The UR3 arm must plan and execute collision-free trajectories from a home configuration to grasp poses, execute a stable grasp using the Robotiq gripper, and transfer the object to a target location within the constraints of the arm workspace and obstacle geometry.

System integration problem: The AGV navigation and arm manipulation sub-systems must share a common ROS 2 communication architecture and a unified mission logic without introducing race conditions or unsafe velocity commands.

Simulation-to-physical conversion problem: The simulation architecture must minimise the code changes required for physical hardware deployment, with hardware interface abstraction separating control logic from sensor and actuator drivers.

2.2.3 Design Constraints and Requirements

The system design is governed by the following constraints: the warehouse floor layout is fixed with known shelf positions; navigation must be deterministic and certifiably safe; the arm must operate within a 500 mm reach radius; the system must run on a Raspberry Pi 4 (AGV) and operator PC (arm planning); total system cost must not exceed $1,300 for a production build; and all software must be open-source and based on ROS 2.

3. Modelling, Solution Methodology, and System Design

3.3.1 Overall System Architecture

The ATLAS Smart Warehouse AGV system is structured as two tightly coupled subsystems operating within a unified ROS 2 communication framework. The AGV subsystem handles mobile navigation and mission management; the Arm subsystem handles object manipulation at target locations.

The AGV subsystem follows a layered architecture: (1) Perception Layer — line sensor, IMU, RFID, odometry; (2) Control Layer — line follower, turn controller, velocity arbiter; (3) Planning Layer — mission FSM, queue manager; (4) Actuation Layer — differential drive (sole `cmd_vel` publisher); and (5) Interface Layer — PyQt5 GUI and CLI tools.

The Arm subsystem follows a five-layer architecture: (1) Physics/Hardware Layer — Gazebo Harmonic simulation; (2) Control Layer — ros2_control with JointTrajectoryController and GripperActionController at 500 Hz; (3) Motion Planning Layer — MoveIt 2 with OMPL and PILZ planners; (4) Task Planning Layer — MoveIt Task Constructor (MTC); and (5) Application Layer — Python and C++ task nodes.

3.3.2 ROS 2 Package Structure

The system comprises 11 ROS 2 packages as described in Table 1.

Table 3.1: ROS 2 Package Structure

Package	Build Type	Purpose
atlas_interfaces	ament_cmake	Custom AGV message definitions
atlas_description	ament_cmake	AGV URDF/Xacro model
atlas_gazebo	ament_cmake	AGV Gazebo world file
atlas_navigation	ament_python	AGV sensor and controller nodes
atlas_mission_manager	ament_python	Mission FSM, GUI, CLI
atlas_bringup	ament_cmake	Master AGV launch file
ur_description	ament_cmake	UR3 URDF/Xacro robot model
ur_gazebo	ament_cmake	UR3 Gazebo simulation launch
moveit_config	ament_cmake	MoveIt 2 SRDF, kinematics, OMPL config
ur_mtc_pick_place_demo	ament_cmake	MTC pick-and-place pipeline
ur_interfaces	ament_cmake	Custom arm message definitions

3.3.3 AGV Mechanical Design

3.3.3.3.3.1 Physical Parameters

The ATLAS AGV physical parameters as defined in `atlas_agv.urdf.xacro` are summarised in Table 2.

Table 3.2: AGV Physical Parameters

Parameter	Value	Description
Body ($B_X \times B_Y \times B_Z$)	$0.30 \times 0.25 \times 0.10$ m	Chassis footprint
Total mass	2.5 kg	Robot mass
Wheel radius (r)	0.05 m	Drive wheel size
Wheel track (L)	0.30 m	Distance between wheels
Wheel width	0.04 m	Tyre width
Caster radius	0.025 m	Passive support wheel
Drive type	Differential drive	Two powered wheels + caster

3.3.3.3.3.2 Real-World Chassis

The physical chassis uses 6061 aluminium extrusion (20×20 mm T-slot) with a 300×250 mm aluminium base plate (3 mm thick), 3 mm L-bracket motor mounts, and an acrylic or aluminium top deck for electronics. The centre of gravity is maintained above the drive axle by placing the battery directly above the wheel axis to maximise traction and stability.

3.3.4 Differential Drive Kinematics

3.3.4.3.4.1 Forward Kinematics

Given left wheel velocity v_L and right wheel velocity v_R, the robot's linear and angular velocities are:

$$v = \frac{v_R + v_L}{2} \tag{3.1}$$

$$\omega = \frac{v_R - v_L}{L} \tag{3.2}$$

The robot pose evolves according to:

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \omega \tag{3.3}$$

3.3.4.3.4.2 Inverse Kinematics

Given desired robot velocity (v, ω), the wheel velocities are computed as:

$$v_L = v - \frac{\omega \cdot L}{2} \tag{3.4}$$

$$v_R = v + \frac{\omega \cdot L}{2} \tag{3.5}$$

Converting to wheel angular velocities:

$$\omega_L = \frac{v_L}{r}, \qquad \omega_R = \frac{v_R}{r} \tag{3.6}$$

3.3.4.3.4.3 Instantaneous Centre of Rotation

The ICR distance from the robot centre is:

$$R = \frac{L}{2} \cdot \frac{v_R + v_L}{v_R - v_L} \tag{3.7}$$

Special cases: straight-line motion ($v_R = v_L$, $R \to \infty$); spin-in-place ($v_R = -v_L$, $R = 0$); pivot about one wheel ($v_L = 0$, $R = L/2$).

3.3.4.3.4.4 Odometry Integration

Position is computed by dead reckoning using midpoint integration (second-order accuracy):

$$\Delta s = \frac{\Delta s_R + \Delta s_L}{2}, \qquad \Delta\theta = \frac{\Delta s_R - \Delta s_L}{L} \tag{3.8}$$

$$x_{k+1} = x_k + \Delta s \cdot \cos\left(\theta_k + \frac{\Delta\theta}{2}\right) \tag{3.9}$$

$$y_{k+1} = y_k + \Delta s \cdot \sin\left(\theta_k + \frac{\Delta\theta}{2}\right) \tag{3.10}$$

$$\theta_{k+1} = \theta_k + \Delta\theta \tag{3.11}$$

3.3.4.3.4.5 Inertia Tensor

For the rectangular chassis (uniform density, $m = 2.5\,\text{kg}$, $0.30 \times 0.25 \times 0.10\,\text{m}$):

$$I_{xx} = \frac{m(b^2 + c^2)}{12} = \frac{2.5(0.25^2 + 0.10^2)}{12} = 0.01510\ \text{kg} \cdot \text{m}^2 \tag{3.12}$$

$$I_{yy} = \frac{m(a^2 + c^2)}{12} = \frac{2.5(0.30^2 + 0.10^2)}{12} = 0.02083 \text{ kg} \cdot \text{m}^2 \tag{3.13}$$

$$I_{zz} = \frac{m(a^2 + b^2)}{12} = \frac{2.5(0.30^2 + 0.25^2)}{12} = 0.03177 \text{ kg} \cdot \text{m}^2 \tag{3.14}$$

3.3.5 UR3 Robotic Arm Kinematic Model

3.3.5.3.5.1 UR3 Denavit-Hartenberg Parameters

The UR3 is a 6-DOF collaborative serial manipulator with a payload rating of 3 kg and a reach of 500 mm. Its modified DH parameters are given in Table 3.

Table 3.3: UR3 Denavit-Hartenberg Parameters (Modified DH Convention)

Joint	a **(m)**	d **(m)**	α **(rad)**	θ **offset**
1 (shoulder_pan)	0.0	0.1519	$\pi/2$	0
2 (shoulder_lift)	-0.24365	0.0	0	0
3 (elbow)	-0.21325	0.0	0	0
4 (wrist_1)	0.0	0.11235	$\pi/2$	0
5 (wrist_2)	0.0	0.08535	$-\pi/2$	0
6 (wrist_3)	0.0	0.0819	0	0

The forward kinematics chain is computed as:

$$T^6_{\text{base}} = T^1_0 \cdot T^2_1 \cdot T^3_2 \cdot T^4_3 \cdot T^5_4 \cdot T^6_5 \tag{3.15}$$

where each transformation follows the standard DH product:

$$T^n_{n-1} = \text{Rot}_z(\theta_n) \cdot \text{Trans}_z(d_n) \cdot \text{Trans}_x(a_n) \cdot \text{Rot}_x(\alpha_n) \tag{3.16}$$

3.3.5.3.5.2 Robotiq 2F-85 Gripper

The Robotiq 2F-85 is an adaptive parallel two-finger gripper with an 85 mm stroke. In simulation, finger coupling is modelled via URDF mimic joints. Only `finger_joint` (range 0.0–0.8 rad) receives commanded positions; all other gripper joints follow through the mimic mechanism. The GripperActionController position range is 0.0 rad (fully open, 85 mm gap) to 0.8 rad (fully closed, 0 mm gap).

3.3.6 Sensor Systems

3.3.6.3.6.1 Line Following Sensor Array

The 8-channel infrared reflectance sensor array computes lateral displacement error using a weighted average:

$$e(t) = \frac{\sum_{i=1}^{8} w_i \cdot s_i}{\sum_{i=1}^{8} s_i} \tag{3.17}$$

where $w_i = [1.0, 0.71, 0.43, 0.14, -0.14, -0.43, -0.71, -1.0]$ and $s_i \in \{0, 1\}$. Binary thresholding: $s_i = 1$ if sensor distance to tape $d_i \leq 0.04$ m, else $s_i = 0$. Junction detection fires when $\sum s_i \geq 5$ for three consecutive frames at a minimum of 2.0 s since the last junction. Sensor specifications are given in Table 4.

Table 3.4: Sensor Specifications

Sensor	Model	Interface	Key Parameter
Line array	QTR-8A	SPI via MCP3008	8 channels, 9.5 mm spacing
IMU	BNO055	I2C	100 Hz, 9-axis, $\pm 0.5^\circ$
RFID	RDM6300	UART	125 kHz, 5–10 cm range

3.3.6.3.6.2 IMU Sensor

Yaw angle ψ is extracted from the IMU quaternion using the standard ZYX Euler formula:

$$\psi = \arctan\left(\frac{2(q_w q_z + q_x q_y)}{1 - 2(q_y^2 + q_z^2)}\right) \tag{3.18}$$

3.3.6.3.6.3 RFID Detection with Hysteresis

The system employs 21 RFID tags: 1 home tag at $(0,0)$ and 20 shelf tags at (x_s, y_a) where $x_s \in \{1,2,3,4\}$ m and $y_a \in \{2,4,6,8,10\}$ m. The hysteresis detection model is:

$$\text{detect:} \quad \sqrt{(x - x_{\text{tag}})^2 + (y - y_{\text{tag}})^2} \leq 0.5 \text{ m} \quad \text{AND armed} \tag{3.19}$$

$$\text{rearm:} \quad \sqrt{(x - x_{\text{tag}})^2 + (y - y_{\text{tag}})^2} > 0.8 \text{ m} \tag{3.20}$$

The 0.3 m hysteresis gap prevents oscillatory detection at the boundary.

3.3.7 Control Systems Design

3.3.7.3.7.1 Line Following PD Controller

A Proportional-Derivative controller is employed for line tracking. The discrete PD control law at 50 Hz is:

$$u_k = K_p \cdot e_k + K_d \cdot (e_k - e_{k-1}) \cdot f_s \tag{3.21}$$

with $K_p = 0.6$, $K_d = 0.2$, and $f_s = 50$ Hz. The output u becomes `angular.z` in the ROS 2 Twist command. The integral term is set to zero ($K_I = 0$) to eliminate windup at junction transitions where the setpoint changes rapidly and no steady-state error exists due to direct position measurement.

Stability Analysis

The closed-loop characteristic equation is:

$$s^2 + 6s + 3 = 0 \tag{3.22}$$

with roots $s = -0.55$ and $s = -5.45$ (both real and negative: stable, overdamped). Natural frequency: $\omega_n = \sqrt{3} = 1.73$ rad/s. The 2% settling time is $t_s \approx 4/0.55 = 7.3$ samples = 0.15 s.

3.3.7.3.7.2 Turn Controller

A bang-bang controller with IMU feedback executes 90° and 180° turns. A constant angular velocity is applied until the target heading is reached:

$$\omega_{\text{cmd}} = \text{sign}(\Delta\theta) \times 0.4 \text{ rad/s} \tag{3.23}$$

The stop condition is $|\theta_{\text{target}} - \theta_{\text{current}}| < 3°$. Theoretical turn durations: 90° turn: $t = \pi/(2 \times 0.4) = 3.93$ s; 180° turn: $t = \pi/0.4 = 7.85$ s.

3.3.7.3.7.3 Motor Velocity PID

Each drive motor requires a velocity PID with initial parameters: $K_p = 2.0$, $K_i = 1.0$, $K_d = 0.05$. Tuning targets: steady-state error $< 2\%$; step-response overshoot $< 10\%$.

3.3.8 Mission State Machine

The complete mission lifecycle is governed by a 12-state Finite State Machine as summarised in Table 5. The Velocity Arbiter ensures that only one velocity source reaches the wheels at any time:

$$\text{output} = \begin{cases} \text{nav\_vel} & \text{if state} \in \text{navigation states} \\ \text{turn\_vel} & \text{if state} \in \text{turning states} \\ \text{dock\_velocity} & \text{if state} \in \text{docking states} \\ \mathbf{0} & \text{if E-stopped (safety override)} \end{cases} \tag{3.24}$$

Table 3.5: Mission State Machine Summary

State	Purpose	Velocity Source
IDLE	Waiting for queued mission	Zero
NAV_SPINE	Following spine north	Line follower
TURNING	90° turn into aisle	Turn controller
NAV_AISLE	Following aisle east	Line follower
AT_SHELF	Arrived at target shelf	Zero
PICKUP	Simulated pick (2 s)	Zero
PIVOT	180° turn	Turn controller
RET_AISLE	Following aisle west	Line follower
RET_TURN	90° turn onto spine	Turn controller
RET_SPINE	Following spine south	Line follower
DOCKED	Arrived at home dock	Zero → Docking FSM
ERROR	E-Stop active	Zero

3.3.9 UR3 Arm Motion Planning

3.3.9.3.9.1 MoveIt 2 Architecture

The arm motion planning follows a five-layer architecture. The URDF/SRDF pair defines robot geometry and semantic collision groupings. The ros2_control layer provides real-time position command interfaces at 500 Hz. MoveIt 2's `move_group` node manages the planning scene, kinematic solvers, and planner plugins. The MoveIt Task Constructor (MTC) provides compositional multi-stage task specification. The application layer dispatches planning requests to the MTC pipeline.

3.3.9.3.9.2 Pick-and-Place Workflow

The complete pick-and-place operation is structured in five phases:

Phase 1 — Initialisation: Gazebo spawning, controller activation, MoveIt 2 start, planning scene population with collision geometry, arm moved to home configuration.

Phase 2 — Pre-Grasp: Gripper commanded to OPEN state (0.0 rad); arm planned via OMPL

to pre-grasp approach pose (100 mm above target object).

Phase 3 — Grasp: Arm descends linearly to grasp pose using PILZ LIN Cartesian planner (10 mm step size); gripper commanded to CLOSED state (0.8 rad); object attached to gripper in planning scene.

Phase 4 — Lift and Transfer: Arm linearly lifted above table (PILZ LIN); arm planned to pre-place pose via OMPL; arm descends to place pose via PILZ LIN.

Phase 5 — Place and Retreat: Gripper commanded to OPEN state; object detached from planning scene; arm retreats and returns to home configuration.

3.3.9.3.9.3 Planner Selection Rationale

OMPL RRTConnect is used for free-space arm motions (probabilistically complete, empirically fastest for 6-DOF configurations). PILZ LIN is used for Cartesian approach and retreat phases (deterministic, guaranteed straight-line, suitable for industrial certification). This dual-planner strategy provides optimal coverage of both free-space and Cartesian motion requirements within a single unified pipeline.

3.3.10 Hardware Architecture

3.3.10.3.10.1 Computing Platform

The Raspberry Pi 4 (4 GB) is selected as the AGV on-board computer (no GPU/SLAM/vision required for AGV navigation). The arm planning runs on an operator PC. Platform comparison is given in Table 6.

Table 3.6: AGV Computing Platform Comparison

Platform	CPU	RAM	ROS 2	Price
Raspberry Pi 4 (4 GB)	4×1.5 GHz	4 GB	Yes	$55
Raspberry Pi 5 (8 GB)	4×2.4 GHz	8 GB	Yes	$80
Jetson Nano	4×1.4 GHz+GPU	4 GB	Yes	$150

3.3.10.3.10.2 Power System

The AGV power budget is shown in Table 7. A 3S LiPo battery (11.1 V, 5000 mAh) provides approximately 1.5 hours of continuous runtime.

Table 3.7: AGV Power Budget

Component	Voltage	Current	Power
2× DC motors (loaded)	12 V	1.0 A each	24 W
Raspberry Pi 4	5 V	2.5 A	12.5 W
All sensors	3.3 V	0.2 A	0.66 W
Motor controller (logic)	5 V	0.05 A	0.25 W
Total			**≈37 W**

3.3.10.3.10.3 Simulation-to-Physical Component Conversion

The complete conversion from simulation to physical hardware is documented in Table 8.

Table 3.8: Simulation-to-Physical Component Conversion

Simulation	Real Hardware	Cost	Integration
Gazebo diff_drive	L298N + DC motors + encoders	$50	GPIO PWM
URDF geometry	Aluminium frame	$60	Fabrication
Virtual line sensor	QTR-8A + MCP3008	$20	SPI
Virtual IMU	BNO055	$30	I2C
Virtual RFID	RDM6300 + 25 tags	$30	UART
ROS 2 on x86	ROS 2 on RPi 4	$70	microSD
Simulated power	3S LiPo + BMS + Buck	$50	Wiring
Gazebo floor	Real floor + black tape	$20	Installation
GUI (PyQt5)	Same GUI on operator PC	$0	WiFi DDS
Total		**≈$330**	

3.3.10.3.10.4 GPIO Pin Mapping

The Raspberry Pi 4 GPIO pin assignment is given in Table 9.

Table 3.9: Raspberry Pi 4 GPIO Pin Mapping

GPIO	Function	Direction	Notes
2	I2C SDA (IMU)	Bidirectional	3.3 V level
3	I2C SCL (IMU)	Output	3.3 V level
5	Encoder L-A	Input	Interrupt
6	Encoder L-B	Input	Interrupt
8	SPI CE0 (ADC)	Output	Active low
9	SPI MISO	Input	—
10	SPI MOSI	Output	—
11	SPI CLK	Output	1.35 MHz
12	PWM0 (left motor)	Output	10 kHz
13	PWM1 (right motor)	Output	10 kHz
14	UART TX (RFID)	Output	9600 baud
15	UART RX (RFID)	Input	9600 baud
17	Motor L-IN1	Output	Direction
22	Motor R-IN3	Output	Direction
23	Motor R-IN4	Output	Direction
24	Encoder R-A	Input	Interrupt
25	Encoder R-B	Input	Interrupt
27	Motor L-IN2	Output	Direction

3.3.11 Software Deployment Architecture

3.3.11.3.11.1 Files Unchanged for Physical Deployment

The following files run without modification on physical hardware: `line_follower.py` (reads and writes ROS topics only); `turn_controller.py` (reads IMU, outputs Twist); `mission_node.py` (pure state machine); `send_mission.py` (CLI publisher); and `atlas_control_center.py` (GUI on operator PC). This represents 70% of the codebase.

3.3.11.3.11.2 New Hardware Interface Nodes

Four simulation plugins are replaced by hardware driver nodes as shown in Table 10.

Table 3.10: Code Changes for Physical Deployment

Simulation File	Replaced By	New File
`libgazebo_ros_diff_drive.so`	motor driver node	`atlas_hardware/motor_driver.py`
`line_sensor.py` (geometric)	SPI ADC reader	`atlas_hardware/line_sensor_hw.py`
`tag_detector.py` (distance)	UART RFID reader	`atlas_hardware/rfid_reader_hw.py`
`libgazebo_ros_imu_sensor.so`	I2C BNO055 driver	`atlas_hardware/imu_hw.py`
`atlas_full.launch.py`	No-Gazebo launch	`atlas_real.launch.py`

3.3.12 GUI and Human-Machine Interface

The PyQt5 control centre uses a dual-threaded architecture. The main Qt thread handles event loop, widget rendering, and user interaction; a background thread runs `rclpy.spin()` and subscription callbacks. Thread-safe communication uses `pyqtSignal`. GUI panels include: Mission Creation, Mission Control (Pause/Resume/Cancel), Emergency Controls (E-Stop, Reset AGV), Robot Status with 11 live telemetry fields, Docking Status, Mission Queue, and Event Log.

4. Results and Discussions with Conclusions

4.4.1 System Performance Metrics

The integrated system was validated through complete mission execution in the Gazebo simulation environment. Key performance metrics are presented in Table 11.

Table 4.1: System Performance Metrics

Metric	Measured Value	Requirement	Status
Line following speed	0.4 m/s	≥ 0.3 m/s	PASS
Turn accuracy	$\pm 3^\circ$	$\pm 5^\circ$	PASS
RFID detection rate	100%	$\geq 95\%$	PASS
Mission completion	100%	$\geq 98\%$	PASS
Docking alignment	± 3 cm, $\pm 3^\circ$	± 5 cm, $\pm 5^\circ$	PASS
E-Stop response time	< 20 ms	< 100 ms	PASS
Mission cycle (S05)	≈ 45 s	< 60 s	PASS
GUI update rate	10 Hz	≥ 5 Hz	PASS
Line tracking accuracy	< 5 mm	< 10 mm	PASS

4.4.2 Mission Timing Analysis

The complete timing for a representative mission (Shelf S05, aisle 5, rack 1) is presented in Table 12.

Table 4.2: Mission Timing Analysis (Shelf S05)

Phase	Distance/Angle	Speed	Time
NAV_SPINE	4.0 m	0.4 m/s	10.0 s
TURNING	$\pi/2$ rad	0.4 rad/s	3.9 s
NAV_AISLE	1.0 m	0.4 m/s	2.5 s
AT_SHELF	—	—	0.5 s
PICKUP	—	—	2.0 s
PIVOT	π rad	0.4 rad/s	7.9 s
RET_AISLE	1.0 m	0.4 m/s	2.5 s
RET_TURN	$\pi/2$ rad	0.4 rad/s	3.9 s
RET_SPINE	4.0 m	0.4 m/s	10.0 s
DOCKED	—	—	1.0 s
Total			**≈44.2 s**

4.4.3 Throughput Estimate

Based on the mission timing data, the single-robot throughput is approximately 80 picks/hour. A fleet of five robots is estimated to achieve approximately 300 picks/hour (75% efficiency due to path conflict avoidance), comparable to the operational throughput of entry-level commercial AMR systems at a fraction of the acquisition cost.

4.4.4 UR3 Arm Performance

The UR3 arm subsystem demonstrated successful pick-and-place operation across all tested configurations. The arm controller operates at 500 Hz, matching the UR robot's typical EtherCAT communication rate. The PILZ LIN Cartesian planner achieved consistent approach and retreat trajectories with a 10 mm step size. MTC compositional planning successfully sequenced nine stages into a coherent mission plan: CurrentState, OpenGripper, Approach, Pick, Lift, Move, Place, Retreat, and CloseGripper. OMPL RRTConnect planning time was non-deterministic but consistently below 2 s for free-space motions.

4.4.5 Comparison with Commercial Systems

Table 13 compares the ATLAS system with established commercial AGV platforms.

Table 4.3: Comparison with Commercial AGV Systems

Feature	**ATLAS (this work)**	**Amazon Robotics**	**MiR 250**	**OTTO 100**
Navigation	Line following	Vision SLAM	LiDAR SLAM	LiDAR SLAM
Robotic arm	UR3 (integrated)	None	None	None
Fleet size	1 (research)	800,000+	Unlimited	Unlimited
Cost	≈$1,250	Proprietary	$25,000	$30,000
Open-source	Yes	No	No	No

The ATLAS system is uniquely distinguished by its integration of an onboard robotic arm for material picking, at approximately 5% of the cost of commercial alternatives.

4.4.6 Challenges and Limitations

4.4.6.4.6.1 Current Limitations

The following limitations were identified: (1) single robot only — no fleet coordination; (2) no obstacle detection — relies on clear pre-planned paths; (3) simulated sensors — no real noise models; (4) no payload physics — object attachment is kinematic; (5) fixed world — cannot handle dynamic warehouse layout changes; (6) odometry drift — acceptable due to RFID resets but accumulates on longer paths; (7) hardcoded pick poses — requires a perception pipeline for generalised picking; and (8) fragile bootstrap timing for controller initialisation.

4.4.6.4.6.2 Design Trade-offs

Key design trade-offs are summarised in Table 14.

Table 4.4: Design Trade-offs Summary

Decision	Benefit	Cost
Line following vs SLAM	Simple, deterministic	Fixed paths only
Junction counting	No map needed	Cannot skip junctions
Bang-bang turns	Simple, reliable	Slightly imprecise
Single velocity arbiter	Safety guaranteed	Complex state machine
$K_I = 0$ (line follower)	No windup	Slight error on curves
GUI separate process	Crash-safe	WiFi dependency
Position command (arm)	Simple control	No force compliance
KDL default IK solver	No extra dependencies	Slower than TracIK

4.4.7 Future Scope

Based on the results and limitations identified above, the following extensions are proposed:

1. **Multi-robot fleet coordination** — fleet manager with traffic rules and deadlock prevention.

2. **LiDAR integration** — obstacle detection and dynamic safety zones.

3. **Computer vision for arm** — Intel D435 depth camera with YOLO-v8 object detection and point cloud grasp pose estimation replacing hardcoded pick coordinates.

4. **Reinforcement learning** — SAC-based policy trained in MuJoCo with Gazebo transfer for adaptive grasping.

5. **LLM-based task planning** — natural language mission dispatch via a locally hosted LLM (e.g., LLaMA3) integrated with the MTC pipeline.

6. **Digital twin** — real-time simulation-to-physical synchronisation for remote monitoring.

7. **Force/torque sensing** — compliant grasping and contact detection using a wrist F/T sensor.

8. **Battery management** — real charge monitoring and autonomous docking for recharging.

4.4.8 Conclusions

4.4.8.4.8.1 Achievements

The ATLAS Smart Warehouse AGV with integrated UR3 robotic arm successfully demonstrates: (1) complete autonomous mission execution with zero human intervention; (2) robust line-following navigation with sub-5 mm tracking accuracy at 0.4 m/s; (3) reliable junction-based wayfinding with RFID position confirmation at 21 warehouse locations; (4) a professional 12-state mission FSM with docking recovery and emergency stop; (5) industrial-quality safety through E-Stop, velocity arbiter, and docking alignment verification; (6) a modern ROS 2 architecture across 11 packages; (7) successful UR3 pick-and-place using MoveIt 2 and MTC compositional planning; (8) integration of OMPL and PILZ planners for optimal motion coverage; (9) a PyQt5 control centre with real-time telemetry; and (10) a documented simulation-to-physical pathway with 70% code reuse.

4.4.8.4.8.2 Contributions

The primary engineering contributions are: a complete open-source warehouse AGV system integrating mobile navigation with a 6-DOF robotic arm under a unified ROS 2 architecture; the velocity arbiter pattern for safe multi-source velocity management; an RFID-augmented junction counting localisation strategy without SLAM; a documented simulation-to-physical deployment methodology; and integration of MTC-based compositional task planning with PILZ Cartesian planning for reliable arm operation.

4.4.8.4.8.3 Final Assessment

The ATLAS project bridges academic mobile robotics and industrial AGV systems, demonstrating that a complete warehouse automation solution encompassing mobile navigation, RFID localisation, robotic arm manipulation, and an operator GUI can be built with open-source software and commercial off-the-shelf hardware at approximately 5% of the cost of commercial alternatives. The system is production-ready in control logic and requires only hardware interface node replacement for physical robot deployment.

References

[1] De Ryck M, Versteyhe M, Debrouwere F, 2020, Automated guided vehicle systems, state-of-the-art control algorithms and techniques, *Journal of Manufacturing Systems*, vol. 54, pp. 152–173.

[2] Azadeh K, De Koster R, Roy D, 2019, Robotized and automated warehouse systems: review and recent developments, *Transportation Science*, vol. 53, no. 4, pp. 917–945.

[3] Siegwart R, Nourbakhsh I R, Scaramuzza D, 2011, *Introduction to Autonomous Mobile Robots*, 2nd edn, MIT Press, Cambridge, MA.

[4] Ogata K, 2010, *Modern Control Engineering*, 5th edn, Prentice Hall, Upper Saddle River, NJ.

[5] Open Robotics, 2023, ROS 2 Humble Hawksbill Documentation, Available at: https://docs.ros.org/en/humble (Accessed: May 2026).

[6] Finkenzeller K, 2010, *RFID Handbook: Fundamentals and Applications in Contactless Smart Cards, Radio Frequency Identification and Near-Field Communication*, 3rd edn, Wiley, Chichester.

[7] Corke P, 2017, *Robotics, Vision and Control*, 2nd edn, Springer, Cham.

[8] Dudek G, Jenkin M, 2010, *Computational Principles of Mobile Robotics*, 2nd edn, Cambridge University Press, Cambridge.

[9] Wurman P R, D'Andrea R, Mountz M, 2008, Coordinating hundreds of cooperative, autonomous vehicles in warehouses, *AI Magazine*, vol. 29, no. 1, pp. 9–20.

[10] Franklin G F, Powell J D, Emami-Naeini A, 2015, *Feedback Control of Dynamic Systems*, 7th edn, Pearson, Harlow.

[11] Siciliano B, Khatib O (eds), 2016, *Springer Handbook of Robotics*, 2nd edn, Springer,

Cham.

[12] Quigley M, Gerkey B, Smart W D, 2015, *Programming Robots with ROS*, O'Reilly Media, Sebastopol, CA.

[13] ISO 3691-4:2020, Industrial trucks — Safety requirements and verification — Part 4: Driverless industrial trucks and their systems, International Organization for Standardization, Geneva.

[14] Pololu Corporation, 2023, QTR-8A Reflectance Sensor Array User's Guide, Available at: https://www.pololu.com/docs/0J13 (Accessed: May 2026).

[15] Bosch Sensortec, 2023, BNO055 Intelligent 9-axis Absolute Orientation Sensor Datasheet, Bosch Sensortec GmbH, Reutlingen.

[16] Thrun S, Burgard W, Fox D, 2005, *Probabilistic Robotics*, MIT Press, Cambridge, MA.

[17] Object Management Group, 2015, Data Distribution Service (DDS) Specification v1.4, OMG Document formal/2015-04-10.

[18] Open Source Robotics Foundation, 2023, Gazebo Classic 11 Documentation, Available at: https://classic.gazebosim.org/ (Accessed: May 2026).

[19] Ollero A, 2005, *Intelligent Mobile Robot Navigation*, Springer, Berlin.

[20] IEC 61496:2020, Safety of machinery — Electro-sensitive protective equipment, International Electrotechnical Commission, Geneva.

A. Complete State Machine Transition Table

Table A1 provides the complete state machine transition table for the ATLAS AGV mission FSM.

Table A.1: Complete AGV FSM Transition Table

Current State	Event / Condition	Next State	Action
IDLE	Queue not empty AND not E-stopped	NAV_SPINE	Pop mission; reset junction count
NAV_SPINE	Junction count == target aisle	TURNING	Publish turn_cmd $(-\pi/2)$
TURNING	turn_done received	NAV_AISLE	—
NAV_AISLE	tag_event matches target	AT_SHELF	—
AT_SHELF	0.5 s elapsed	PICKUP	—
PICKUP	2.0 s elapsed	PIVOT	Publish turn_cmd (π)
PIVOT	turn_done received	RET_AISLE	—
RET_AISLE	Junction detected	RET_TURN	Publish turn_cmd $(+\pi/2)$
RET_TURN	turn_done received	RET_SPINE	—
RET_SPINE	Home tag detected	DOCKED	—
DOCKED	0.5 s elapsed	DOCKING	Enter docking FSM
DOCKING	Position verified	ALIGNMENT	—
ALIGNMENT	Heading + lateral OK	READY	—
READY	0.5 s elapsed	IDLE	Log “SYSTEM READY”
ANY	/atlas/estop received	ERROR	Set estopped=True
ERROR	/atlas/reset received	IDLE	Clear state
ANY	/atlas/reset_to_dock	RESETTING	Gazebo teleport

(Continued)

Current State	Event / Condition	Next State	Action
RESETTING	Callback received	DOCKING	Verify alignment

B. Software Installation and Launch Guide

Prerequisites

```
# Ubuntu 22.04 with ROS 2 Humble (AGV subsystem)
sudo apt install ros-humble-desktop
sudo apt install ros-humble-gazebo-ros-pkgs
sudo apt install python3-pyqt5

# Ubuntu 24.04 with ROS 2 Jazzy (Arm subsystem)
sudo apt install ros-jazzy-desktop
sudo apt install ros-jazzy-moveit
sudo apt install ros-jazzy-moveit-task-constructor-core
```

Build Procedure

```
cd ~/atlas_ws
source /opt/ros/humble/setup.bash
colcon build --symlink-install
source install/setup.bash
```

Launch and Operation

```
# Full AGV simulation
ros2 launch atlas_bringup atlas_full.launch.py

# UR3 arm simulation
ros2 launch ur_gazebo ur.gazebo.launch.py

# Send mission via CLI
ros2 run atlas_mission_manager send_mission S05

# Launch GUI standalone
```

```
ros2 run atlas_mission_manager atlas_gui


# Real-world AGV (no Gazebo)
ros2 launch atlas_bringup atlas_real.launch.py
```

C. Viva Examination Questions Summary

Architecture Questions

1. Why was ROS 2 chosen over ROS 1 for this project?
2. Why is a differential drive configuration used for the AGV?
3. Why is `mission_node` the sole `cmd_vel` publisher?
4. Why was MTC chosen over the direct MoveGroup API for arm control?
5. Why is a PD controller used instead of full PID for line following?

Kinematics Questions

1. Derive the forward kinematics for a differential drive robot from first principles.
2. Calculate the maximum linear speed for a 200 RPM motor with a 50 mm wheel.
3. Derive the UR3 forward kinematics using the DH parameter table provided.
4. What is the Instantaneous Centre of Rotation and when does it lie at infinity?
5. Why is the robot's centre of gravity placed above the drive axle?

Control and Arm Questions

1. Draw and explain the PD control block diagram for line following.
2. Calculate the settling time for the line-following PD controller.
3. Explain why the integral gain is set to zero for the line follower.
4. What is the difference between OMPL and PILZ motion planners?
5. How would physical UR3 hardware deployment differ from the simulation?